MSc Data Science: R for Data Science

PhD Student Raphaël Razafindralambo (Email: raphael.razafindralambo@inria.fr)

Last updated on 2025-11-05

Contents

R Scripts

> RStudio is the preferred integrated development environment (IDE) for writing and running R scripts. Installation steps are detailed in the notebook of the first session.

An **R Script** is a simple text file (usually with the `.R` extension) that contains a sequence of R commands. It allows you to **save**, **reuse**, and **share** your code easily, instead of typing commands directly into the console.

You can create an R Script in RStudio by clicking:
File → New File → R Script
and then run lines of code with:
Ctrl + Enter (Windows) or **Cmd + Enter** (Mac).

Rmarkdown

Rmarkdown is a comprehensive way of creating reports which involve R (or other languages). A synthetic presentation of the Rmarkdown system is available here: https://rmarkdown.rstudio.com/lesson-1.html. For advanced details on the language, please refer to https://bookdown.org/yihui/rmarkdown/.

The basic commands of Rmarkdown:

- the `#` allows to organize the document in sections, subsections, ... The more you add `#`, the more you go down into the subsections, subsubsections, ...
- the `*` allows to higlight some part of the text: if you use one `*tyty*` it produces *tyty*. If you use `**tyty**`, the text **tyty** will be in bold.
- the `>` allows to create some "citation / remark box": > The citation that I would like to highlight!
- It is possible to provide online code using single quote. As an exemple: `ls()`.

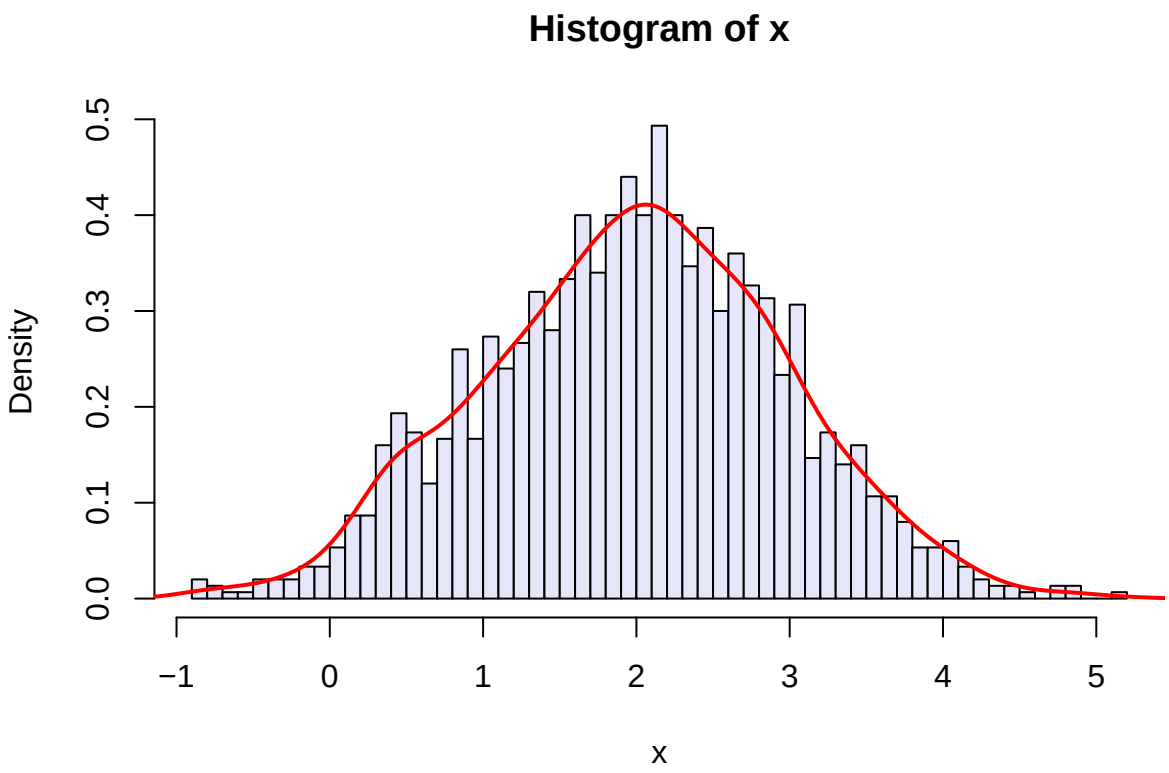
It is also possible to provide parts of code, that are enventually run:

```
rnorm(15)
```

```
##  [1]  0.47811784  2.06105660 -1.29464987 -0.47434357 -1.08406510  1.41358977
##  [7]  0.02104417 -0.93771843  0.64527391 -0.34087128  0.81734934 -1.23943620
## [13]  0.43695302  0.02676722  0.23575230
```

There are several options:

- the first one is the language to use (here, R),
- the `echo` option allows to disable the display of the output. Notice that the code is however run and you can use later its ouptput!



- the `include` option allows to hide both the code and the output (even though the code is run and the output exists in the memory).
- the `eval` option disables the execution of the code.

The different outputs for the Rmd file are either HTML, PDF or Word documents.

Shiny

The `shiny` package for R is a great way to creat interfaces with some R codes. A shiny code creates a web page with JavaScript which allows the user to execute some (hidden) R code and look at the results in a very easy way.

```r
library(shiny)
runExample("01_hello")
```

To start a Shiny app, you have to create two files:

- `ui.R` is devoted to design the user interface,
- `server.R` is running the R codes conditionnaly to the options set by the user through the interface.

The two minimal files ui.R and server.R can be used to start a new app project. For more details on the desiging of Shiny App, please refer to http://shiny.rstudio.com/tutorial/.

Projects

This project aims to **explore and understand real-world applications of R** through small, hands-on experiments on statistical methods and data analysis.
You will learn not only to *implement* statistical tools, but also to *explain them pedagogically* to others.

The focus is both **technical** and **educational**:
you should be able to demonstrate that you understand *how* the method works, *how to implement it in R*, and *how to explain it clearly*.

Objectives

- Learn to implement a method in R, **both manually and using built-in packages**.
- Apply your implementation to **"real" datasets** if possible.
- Understand and **explain step by step** how the method works in R.
- Prepare a short, clear, and pedagogical presentation of your results.

Monitoring sessions

There will be **three mandatory sessions** dedicated to the project, during which you will work in groups and receive guidance. This project is **not extensive**; it is designed to be completed during the three sessions.

Final Presentation and Evaluation

The presentations will take place **during the week of December 8**.
Each group will have a short time slot to present their work in front of the class.

You are **free to choose the presentation format**, as long as it is **clear and easy to follow**. For example, you may present directly from your **RMarkdown**, use a few **slides (PowerPoint, PDF)**, or simply **show your analysis interactively with a notebook**.

Each presentation will last **10 minutes**, followed by **5 minutes of questions**.
Questions will be asked by **myself** and by **one other group chosen at random**

Evaluation

Each group will receive **two grades**:

Component	Description	Weight
Presentation	Clarity, pedagogy, correctness of implementation, and quality of explanations	70%
Questions	Quality and relevance of the question asked to another group	30%

Note: The grading scheme may be adjusted if necessary.

List of projects

Each group will be composed of **two students**. Every group must choose one project topic, and no two groups may work on the same project. The assignment rule is *first come, first served*.

A Google Sheet will be shared for you to fill in your **group members** and **chosen project**. You will have **until November 12 (!!)** to form your group and select your topic.

1. Statistical Tests of Independence Explore how to test whether two categorical variables are statistically independent.
Use methods such as the **Chi-squared test** (between others) and interpret the results.

---

2. Statistical Student Tests Compare the means of one or two groups using **Student's t-tests**. Implement the t-statistic manually and compare results with the `t.test()` function in R.

---

3. Statistical Tests for Benchmarking (McNemar's Test) Use **McNemar's test** to compare the performance of two classifiers on the same dataset. It is particularly useful in **machine learning** for comparing confusion matrices on paired binary outcomes.

---

4. Empirical CDF and Kolmogorov Tests Study the **empirical cumulative distribution function (ECDF)** and the **Kolmogorov–Smirnov test**.
Test whether a dataset follows a given distribution, or whether two samples come from the same one. Visualize ECDFs (implementated by yourself) and interpret.

---

5. K-Means Clustering Implement the **k-means algorithm** from scratch and compare it with R's `kmeans()` function.
Apply it to a dataset to discover natural clusters and visualize the partitioning.
Discuss initialization effects and convergence. It is also worth exploring the K-means++ initialization.

---

6. Principal Component Analysis (PCA) Perform and analyse **dimensionality reduction** using PCA.
Implement PCA step by step using covariance matrices and eigenvalues and reproduce the same analysis with a standard R package.

---

7. Linear Regression Study **linear regression** models on real-world datasets.
You may use `lm()` to train your model. Use diagnostic plots to assess model fit and residual assumptions.

---

8. Deep Learning in R with torch Explore the basics of **deep learning** using the `torch` package in R.
Implement a simple neural network for regression or classification.
Experiment with training, loss functions, and visualizing learning curves.

---

9. Bootstrap Method (Confidence Intervals) Use **bootstrap resampling** to estimate confidence intervals for means, medians, regression coefficients, ... Implement the procedure manually and compare with R's `boot` package.
Interpret variability and bias through simulations.

---

10. Random Forests Implement and analyze **Random Forests** for classification or regression using the associated R package.
Study the concept of bagging and variable importance.
Optionally, attempt a simple manual implementation of the averaging principle.